CS 671 | Deep Learning

Transformers, BERT, GPT & Large Vision Models

Complete Lecture Notes

Self-Attention | Encoder–Decoder | BERT | GPT | RLHF | ViT | DINO

Dr. Aditya Nigam | IIT Mandi

0. Overview & Learning Objectives

Why Transformers Matter

In the years before 2017, sequential modelling was dominated by RNNs and LSTMs. These architectures process inputs one token at a time, which makes them inherently slow to train and prone to forgetting long-range dependencies. The Transformer, introduced by Vaswani et al. in the seminal paper "Attention Is All You Need" (NeurIPS 2017), replaced recurrence entirely with the self-attention mechanism — a content-based addressing scheme that lets every token in a sequence directly attend to every other token in a single matrix multiplication.

This single architectural idea launched a new era of deep learning. The encoder half gave rise to BERT (2018) and the bidirectional family of language understanding models. The decoder half gave rise to GPT (2018, 2019, 2020, ...) and the autoregressive family of generative language models. Together they power virtually every modern Large Language Model (LLM) — ChatGPT, Claude, Gemini, LLaMA, Mistral, and many more. The same machinery, with patches replacing word tokens, gave rise to Vision Transformers (ViT) and Large Vision Models (LVMs) — DINO, CLIP, DALL·E, GPT-4o, and the rapidly growing world of multimodal foundation models.

After completing these notes you will be able to:

- Explain the Encoder-Decoder architecture of the original Transformer and the role of each component (attention, positional encoding, layer normalization, feed-forward network).
- Derive scaled dot-product self-attention from first principles and walk through the worked example with Q, K, V matrices.
- Distinguish self-attention from cross-attention and explain why the decoder uses masked multi-head attention.
- Describe BERT's pre-training objectives — Masked Language Modeling (MLM) and Next Sentence Prediction (NSP) — and the input embedding decomposition into token + segment + position.
- Explain the GPT family as decoder-only autoregressive language models, including the role of byte-pair encoding (BPE) and top-K sampling.
- Outline the four-stage LLM training pipeline (pretraining → SFT → RM → RL) and the principle of Reinforcement Learning from Human Feedback (RLHF).
- Describe the Vision Transformer (ViT) architecture, the patch-embedding step, and the role of the [CLS] token in classification.
- Contrast supervised vs. self-supervised pretraining and explain the teacher-student / BYOL principle behind DINO.

These notes are written as a stand-alone reference at the IIT graduate-undergraduate level. They assume familiarity with basic deep learning (MLPs, gradient descent, backpropagation) but not with sequence modelling. Wherever a slide presents a figure, the figure is reproduced and discussed in the text.

1. The Transformer Architecture

1.1 The Encoder-Decoder Pattern

At the highest level of abstraction, a Transformer is an Encoder-Decoder model used for sequence-to-sequence (seq2seq) tasks like machine translation. Given an input sentence in a source language (e.g. "I am good") the model produces a target sentence in another language (e.g. the French "Je vais bien"). The encoder ingests the entire source sentence and turns it into a dense numerical representation; the decoder consumes this representation and emits the target sequence one token at a time.

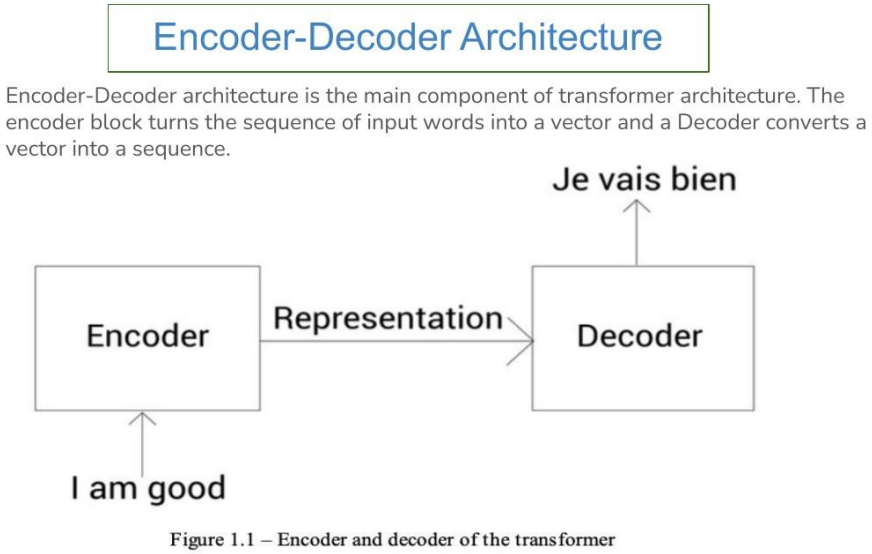

Figure 1.1 — Encoder-Decoder view of the Transformer. The encoder summarises the source sentence "I am good" into a representation; the decoder uses that representation to produce the French translation "Je vais bien".

In practice neither side is a single block — both encoder and decoder are stacks of N identical sub-blocks. The original Transformer used N = 6 on each side. The output of the last encoder is the "representation" that flows into every decoder block.

> **Intuition**
>
> Think of the encoder as a careful reader: it reads the entire source sentence and produces a rich, context-aware vector representation of every input token. Think of the decoder as a writer: starting from a special <SOS> (start-of-sequence) marker, it generates one target word at a time, looking at (a) the encoder's representation and (b) the words it has produced so far. This pattern — read the whole input first, then generate output autoregressively — is the foundation of every Transformer-based seq2seq system.

1.2 Inside an Encoder Block

Each encoder block has two sub-layers. The first is a multi-head self-attention layer that lets every token attend to every other token in the input. The second is a position-wise feed-forward network (FFN) — a

small two-layer MLP applied independently to each token position. Both sub-layers are wrapped with a residual connection and layer normalization (the "Add & Norm" pattern).

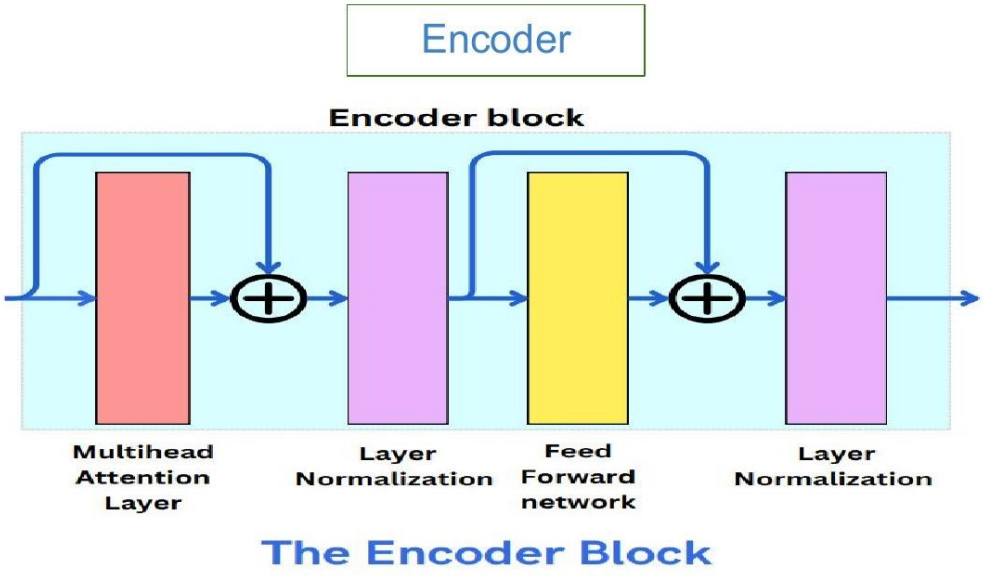

Figure 1.2 — A single encoder block: multi-head attention → Add & Norm → feed-forward → Add & Norm. The blue arcs depict residual (skip) connections that bypass each sub-layer.

1.3 Inside a Decoder Block

Each decoder block has three sub-layers. First, masked multi-head self-attention attends only over previously generated tokens (the mask prevents the model from "looking ahead" at future positions during training). Second, a cross-attention layer (multi-head attention over encoder output) lets each decoder position attend to every encoder position. Third, a position-wise FFN. Each sub-layer is again wrapped in residual + layer-norm.

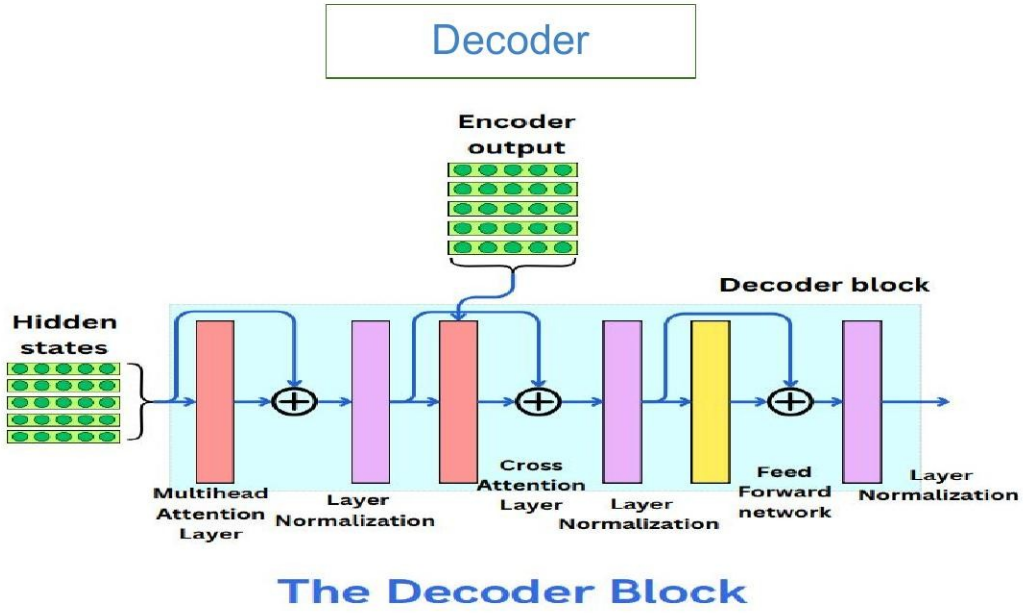

Figure 1.3 — A single decoder block. Three sub-layers: masked self-attention (over decoder hidden states only), cross-attention (queries from decoder, keys/values from encoder output), and a position-wise feed-forward network.

Why is decoder self-attention masked?

During training, the entire target sentence is fed to the decoder at once ("teacher forcing"). Without masking, the decoder could simply look at the next token and copy it — defeating the purpose of generation. The mask sets all attention scores for future positions to $-\infty$ before the softmax, so they receive zero attention weight. At inference time, the model genuinely does not know future tokens; the mask makes training and inference behave consistently.

2. The Attention Mechanism

2.1 What Attention Computes

Attention answers a content-based query. Given a query vector q (representing what we are looking for) and a set of key-value pairs (k_i, v_i), attention returns a weighted sum of the values, where the weight of v_i is determined by how well its key k_i matches the query q. Tokens whose keys closely match the query receive high weight; tokens whose keys do not match receive near-zero weight.

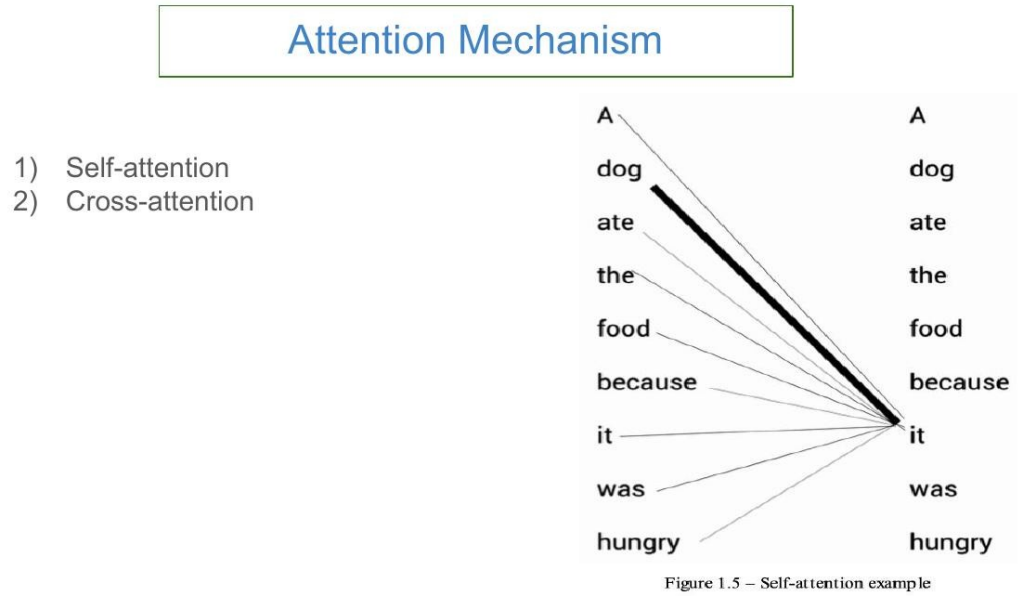

Figure 1.5 – Self-attention example

Figure 2.1 — Self-attention example for the sentence "A dog ate the food because it was hungry." When computing the representation for the word "it", the strongest attention weight goes to "dog" — correctly resolving the pronoun coreference. Line thickness encodes attention weight.

Filing-cabinet analogy

Imagine you are searching for information. You have a query (the topic you want to look up) and a filing cabinet full of folders. Each folder has a label (the key) and contents (the value). You compare your query against each label, get a similarity score, normalise the scores so they sum to 1, and then return a weighted mixture of the folder contents — heavily weighted toward folders whose labels matched your query best. Self-attention does exactly this for every token in a sentence simultaneously.

2.2 Self-Attention vs. Cross-Attention

Both flavours of attention use the same Q-K-V machinery. They differ only in where Q, K, and V come from.

- Self-attention (used inside every encoder block, and in the masked self-attention of the decoder): all of Q, K, V are computed from the same set of hidden states. Each token attends to every other token in the same sequence.

- Cross-attention (used in the second sub-layer of every decoder block): Q comes from the decoder's hidden states; K and V come from the encoder's output. This is how the decoder "looks at" the source sentence while generating the target.

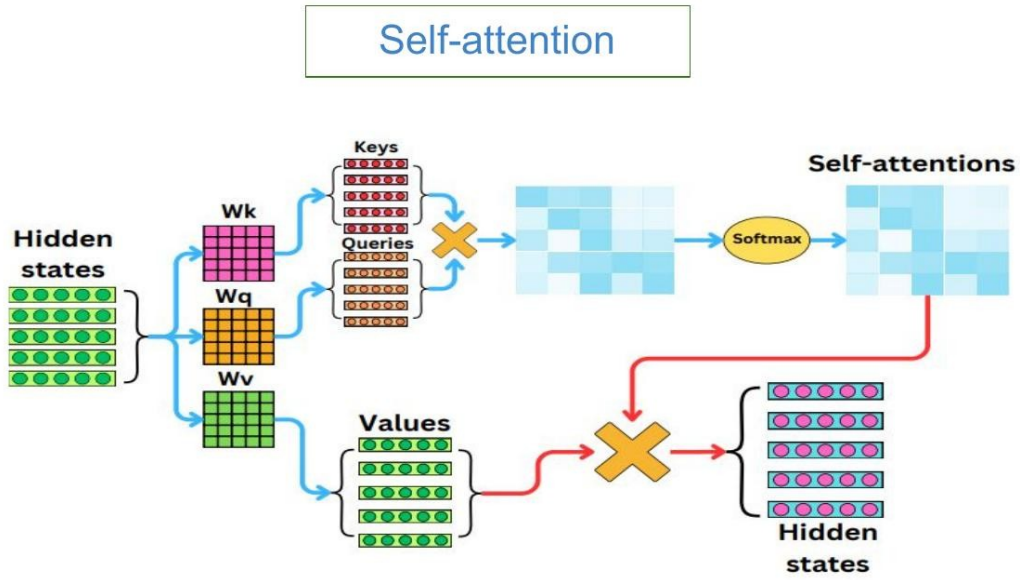

Figure 2.2 — Self-attention. Q, K, V are all derived from the same hidden states by linear projections W_Q, W_K, W_V.

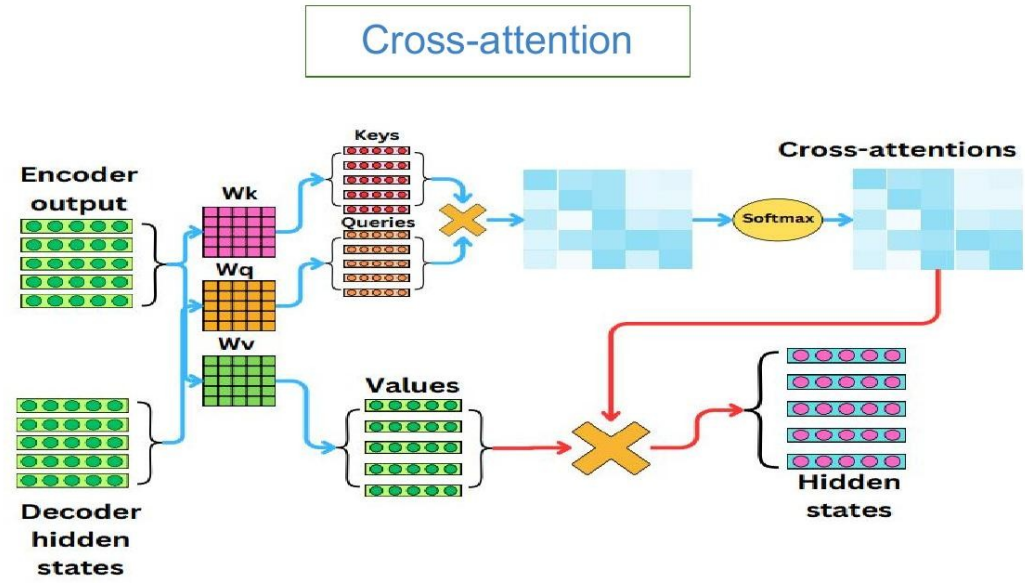

Figure 2.3 — Cross-attention. Queries come from the decoder hidden states; keys and values come from the encoder output.

2.3 The Scaled Dot-Product Attention Formula

Attention is computed in a single matrix operation:

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$$

Here $Q \in \mathbb{R}^{(n \times d_k)}$, $K \in \mathbb{R}^{(n \times d_k)}$, $V \in \mathbb{R}^{(n \times d_v)}$, and n is the sequence length. The product QK^T produces an $n \times n$ matrix of similarity scores. We divide by $\sqrt{d_k}$ for numerical stability (we will see why in Section 2.5), apply a row-wise softmax to get attention weights that sum to 1, and finally multiply by V to produce the $n \times d_v$ output.

2.4 Worked Example: "I am good"

Let's trace the full self-attention computation on a 3-token sentence with embedding dimension 512. The slides walk through this example explicitly — we reproduce it here with mathematical commentary.

Step 1 — Form the input matrix X

Each word is a row of length 512. Stacking the three words, X has shape 3×512:

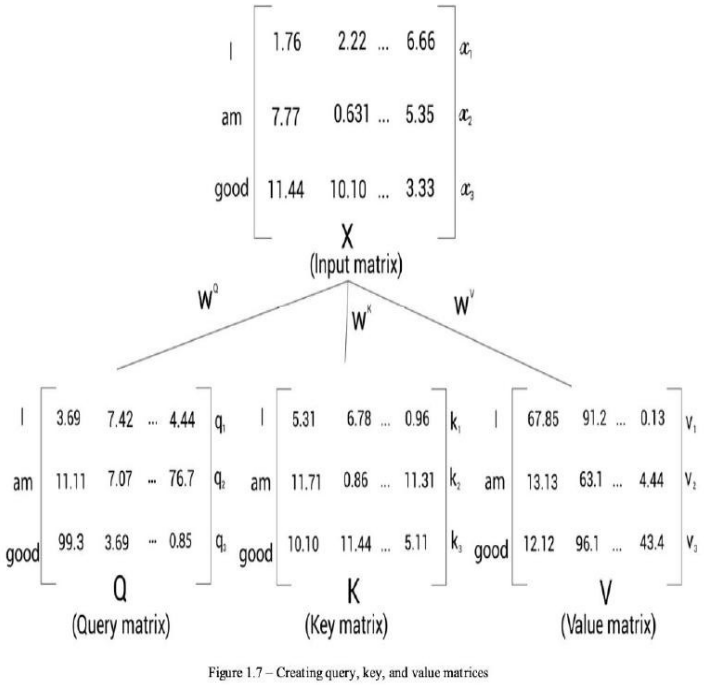

Figure 1.7 – Creating query, key, and value matrices

Figure 2.4 — Input matrix X (top) is projected by three learned weight matrices W^Q, W^K, W^V to produce Q, K, V respectively. Each output matrix has shape 3×d_k.

Step 2 — Compute $Q \cdot K^T$ (similarity scores)

The product QK^T is a 3×3 matrix whose (i, j) entry is the dot product $q_i \cdot k_j$ — the unnormalised similarity between query i and key j.

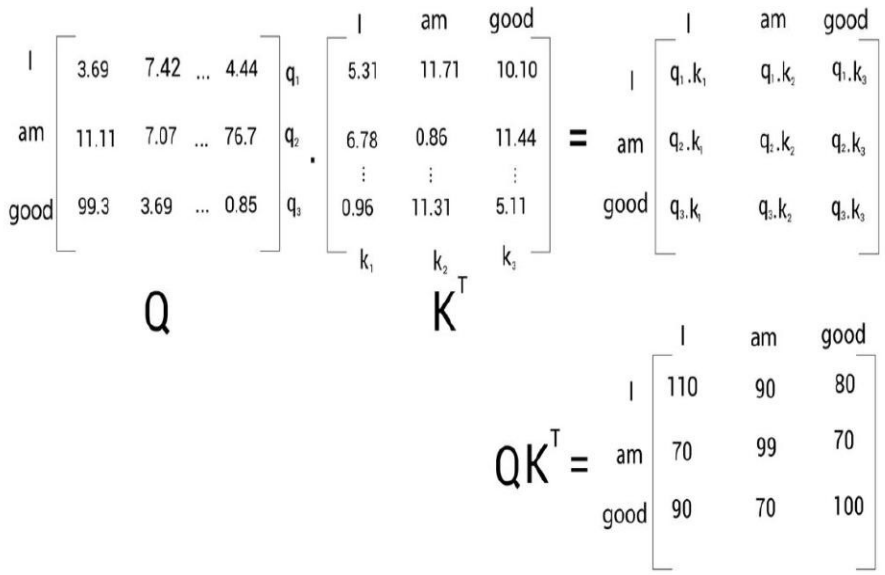

Figure 1.10 – Computing the dot product between the query and key matrices

Figure 2.5 — Computing QK^T. Each entry is a dot product of one query row with one key row. In this example the diagonal dominates (a token is most similar to itself).

Step 3 — Scale by $\sqrt{d_k}$

Divide every entry by $\sqrt{d_k}$. With $d_k = 64$ (a common choice in the original Transformer) the scaling factor is 8.

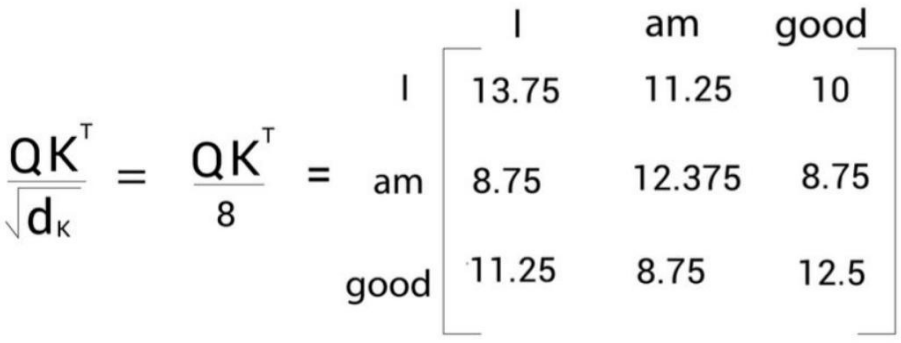

Figure 1.14 – Dividing $Q.K^T$ by the square root of d_k

Figure 2.6 — After dividing by $\sqrt{d_k} = 8$, the scores live in a more moderate range. This prevents the softmax from saturating in extreme regions.

Why divide by $\sqrt{d_k}$?

Suppose the components of Q and K are independent random variables with mean 0 and variance 1. The dot product $q \cdot k = \Sigma_j q_j k_j$ then has variance d_k — i.e. it grows with the dimension. For large d_k

the dot products become very large in magnitude, pushing the softmax into regions where the gradient is essentially zero (saturation). Dividing by $\sqrt{d_k}$ normalises the variance back to 1 and keeps the softmax well-behaved during training. This is one of the most important practical details in the original Transformer paper.

Step 4 — Apply softmax

The softmax is applied row-wise. Each row now sums to 1 and represents the attention distribution of one query over all keys.

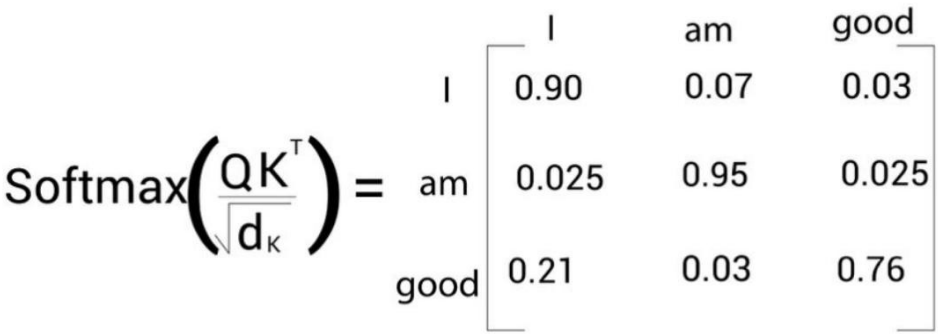

Figure 1.15 – Applying the softmax function

Figure 2.7 — Row-wise softmax. The (i, j) entry now represents the probability that token i should attend to token j. Notice each row sums to 1.

Step 5 — Multiply by V to get the output Z

Each output row z_i is a weighted sum of all value rows, weighted by the i-th attention distribution:

$$Z = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

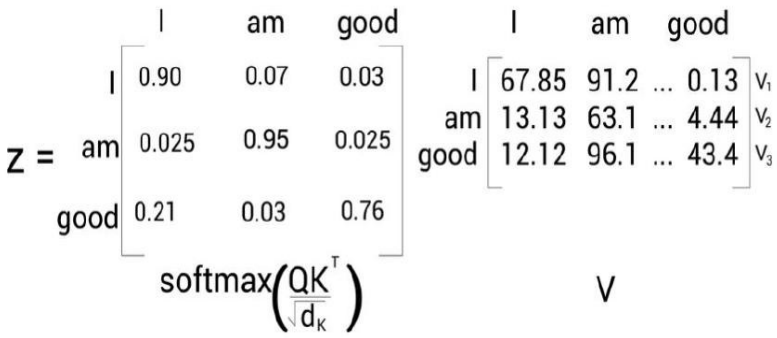

Figure 1.16 – Computing the attention matrix

Say we have the following:

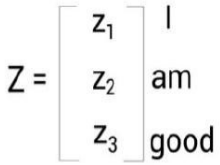

Figure 1.17 – Result of the attention matrix

Figure 2.8 — The final attention output Z = softmax(QK^T/√d_k) ·V. Each row z_i of Z is a context-aware representation of token i — a weighted blend of all value vectors with weights given by attention.

Concretely, the output for the first token ("I") is:

$$z_1 = 0.90 \cdot v_1 + 0.07 \cdot v_2 + 0.03 \cdot v_3$$

So z_1 is dominated by v_1 (the value vector of "I"), with small contributions from "am" and "good". This is why each output token gets a context-aware representation that is mostly itself but enriched with relevant information from other tokens.

2.5 Why "Multi-Head"?

A single attention layer can only learn one type of relationship between tokens (e.g., subject-verb agreement). Multi-head attention runs h independent attention computations in parallel — each with its own learned Q, K, V projections — letting the model attend to information from different representation subspaces simultaneously. The h outputs are concatenated and passed through a final linear layer:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \ldots, \text{head\_h}) \cdot W\hat{}O$$

In the original Transformer paper, h = 8, with each head using $d_k = d_v$ = d_model / h = 64.

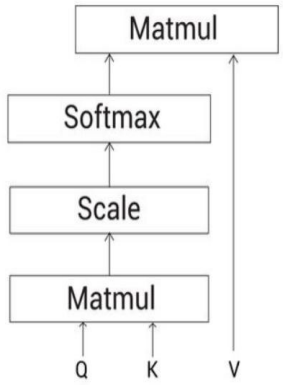

Figure 1.22 – Self-attention mechanism

$$\text{Multi-head attention} = \text{Concatenate}(Z_1, Z_2, \ldots Z_i, \ldots Z_8)W_0$$

Figure 2.9 — Scaled dot-product attention in block form (left): MatMul(Q, K^T) → Scale → Softmax → MatMul with V. Multi-head attention runs h such blocks in parallel and concatenates.

Intuition — what each head learns

Empirically, different heads in a trained Transformer specialise in different linguistic phenomena: one head may track syntactic dependencies (subject-verb agreement), another may track coreference (linking pronouns to their antecedents), another may track positional patterns (next-word, previous-word relations). This is similar in spirit to using multiple convolutional filters in a CNN — each filter learns to detect a different kind of pattern. Concatenating the heads gives the next layer rich, multi-faceted information about every token.

3. Positional Encoding, Layer Norm, and FFN

3.1 Positional Encoding

Self-attention is permutation-equivariant: shuffle the input tokens and the output is shuffled identically — the model has no idea which token came first. Since word order matters in language, we must inject positional information explicitly. The original Transformer adds sinusoidal positional encodings to the input embeddings:

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{(2i/d\_model)}) \quad \text{(even dimensions)}$$
$$\text{PE}(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{(2i/d\_model)}) \quad \text{(odd dimensions)}$$

Here pos is the position of the token in the sequence (0, 1, 2, …) and i is the dimension index inside the embedding. Each dimension of the encoding corresponds to a sinusoid with a different wavelength, ranging from 2π up to 10000·2π.

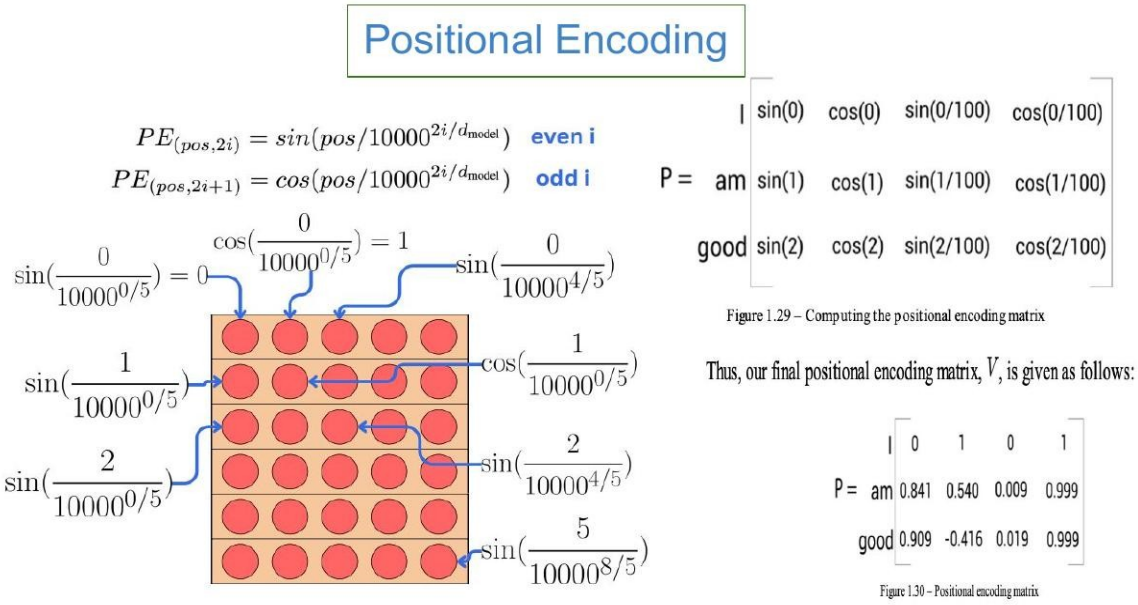

Figure 3.1 — Sinusoidal positional encoding. Left: each dimension i picks a different frequency 1/10000^(2i/d). Right: applied to our 3-token example, position 0 yields (sin0, cos0, …) = (0, 1, …); position 1 yields (sin 1, cos 1, sin 0.01, cos 0.01, …); and so on.

Why sinusoids? Three key properties

- Bounded values: every PE entry lies in [-1, 1], so the encodings won't dominate the embeddings.
- Unique encoding for each position: with d_model dimensions covering different frequencies, no two positions get identical encodings.
- Linear relationships between positions: PE(pos+k) can be written as a linear function of PE(pos), allowing the model to learn relative position relations easily.

Token embeddings and positional encodings are simply added element-wise and fed into the first encoder/decoder block. Many later models (BERT, GPT) use learned positional embeddings instead of sinusoidal ones — but the principle is identical.

3.2 Layer Normalization

Layer Normalization is a per-token, per-feature normalisation that stabilises training. Given a hidden state $x \in \mathbb{R}^d$ at one position, it computes:

$$\mathrm{LN}(x) = \gamma \cdot (x - \mu) \,/\, \sigma \;+\; \beta, \quad \mu = \mathrm{mean}(x), \;\; \sigma = \mathrm{std}(x)$$

γ and β are learned scale and shift parameters of dimension d. Unlike batch normalization, layer norm normalises across the feature dimension within a single example — so it works identically at training and inference and is independent of batch size. This is critical for variable-length sequences.

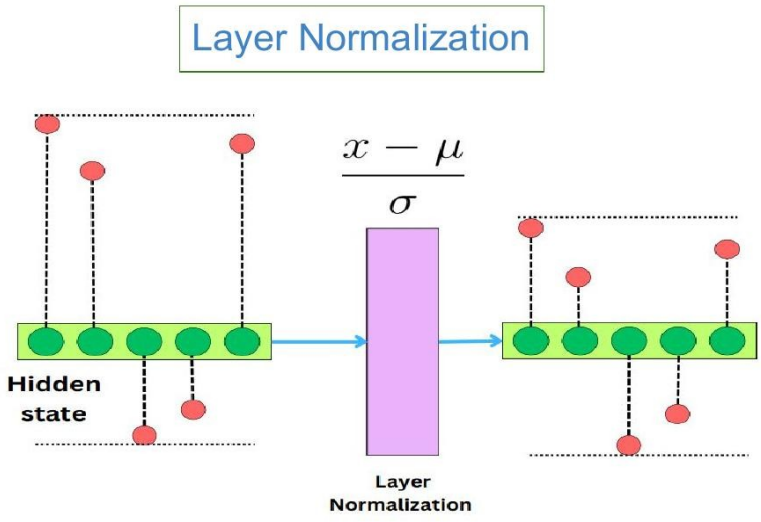

Figure 3.2 — Layer Normalization. The hidden state of one token (left) is normalised to zero mean and unit variance across its features (right) before being passed on.

3.3 Position-wise Feed-Forward Network (FFN)

After attention has mixed information across positions, each token is passed through an identical two-layer MLP — applied independently to every position. This is the "position-wise FFN":

$$\mathrm{FFN}(x) = \max(0,\, x \cdot W_1 + b_1) \cdot W_2 + b_2$$

In the original Transformer d_model = 512 and the inner dimension d_ff = 2048 (i.e. 4× expansion). The first linear layer projects up to 2048, ReLU, then the second linear layer projects back down to 512. The FFN provides per-token nonlinear transformation capacity that complements the (linear in V) attention mixing.

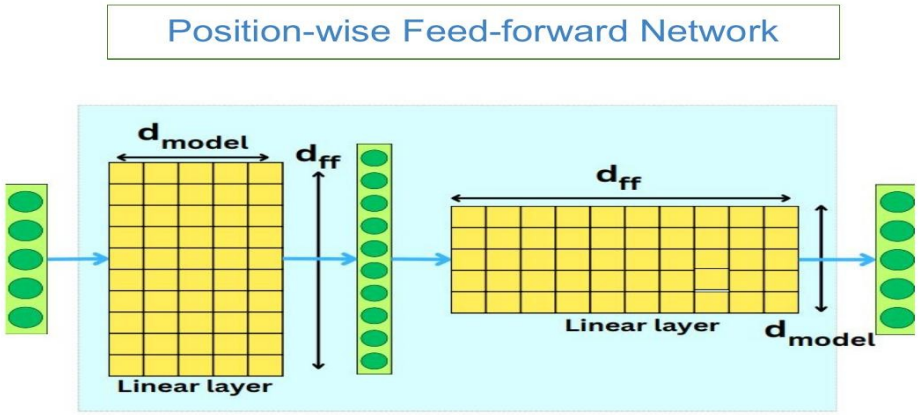

Figure 3.3 — Position-wise FFN. The same two-layer MLP (d_model → d_ff → d_model, with ReLU in between) is applied independently to every token position.

3.4 Residual Connections and the Add-&-Norm Wrapper

Every sub-layer in the Transformer is wrapped as:

$$\text{output} = \text{LayerNorm}(\, x + \text{Sublayer}(x)\,)$$

The residual connection x + Sublayer(x) ensures gradients flow easily through the deep stack (avoiding vanishing-gradient issues), while LayerNorm keeps activations well-scaled. This pattern — present in every encoder and decoder block — is what makes Transformers trainable at depth (24, 48, 96+ layers).

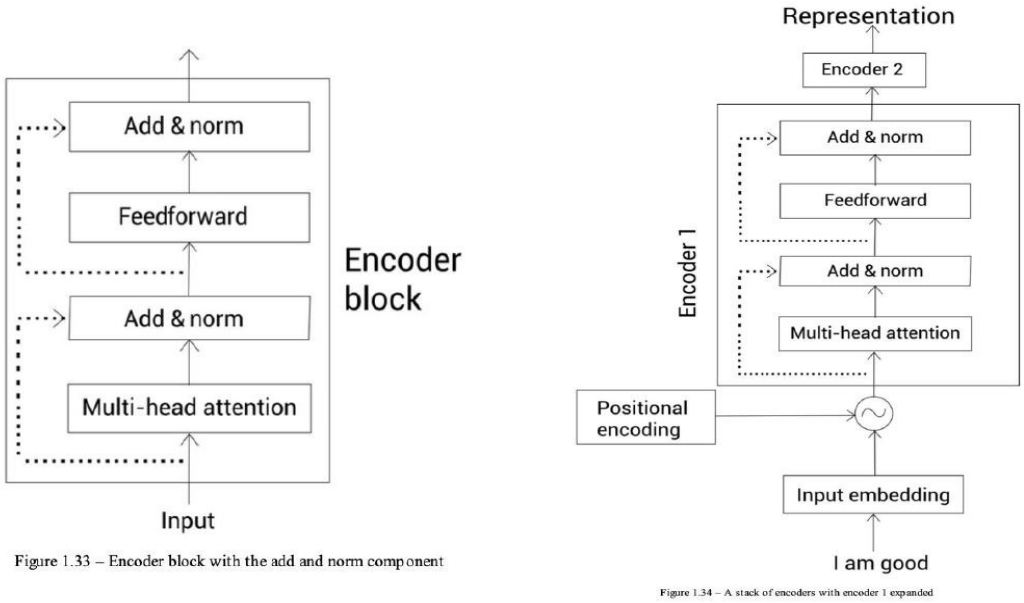

Figure 3.4 — Encoder block with explicit Add & Norm components (left) and a 2-encoder stack with positional encoding shown (right). The dotted arcs are the residual connections.

4. The Complete Transformer Architecture

Stacking everything together, the Transformer for sequence-to-sequence tasks looks as follows. The left half is the encoder stack (N copies of an encoder block), the right half is the decoder stack (N copies of a decoder block), with a Linear-and-Softmax "prediction head" at the top of the decoder.

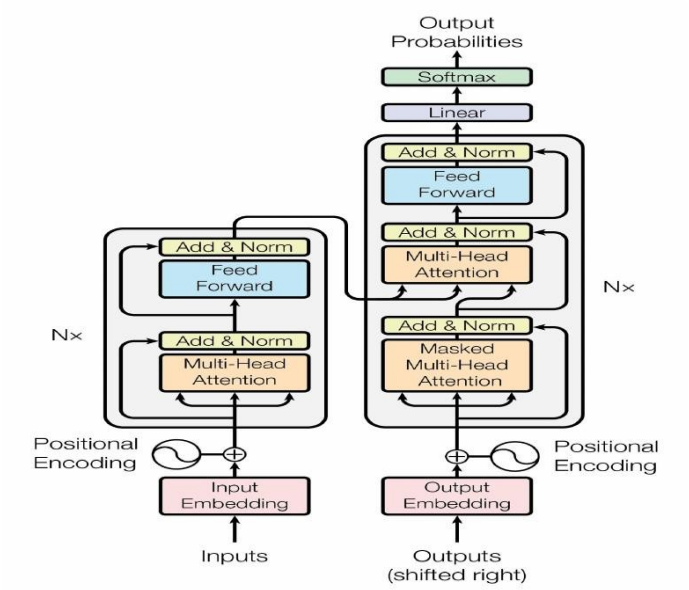

Figure 4.1 — The complete Transformer architecture from Vaswani et al. (2017). Encoder (left): N encoder blocks taking the embedded + positionally-encoded input. Decoder (right): N decoder blocks with masked self-attention, cross-attention to encoder output, and a final Linear → Softmax that produces output token probabilities.

4.1 The Prediction Head

The decoder produces a sequence of hidden state vectors of dimension d_model. The prediction head is a single linear layer of shape d_model × |vocab| followed by a softmax. The output is a probability distribution over the entire vocabulary at every position. To pick the next token we either take the argmax (greedy decoding) or sample from the distribution.

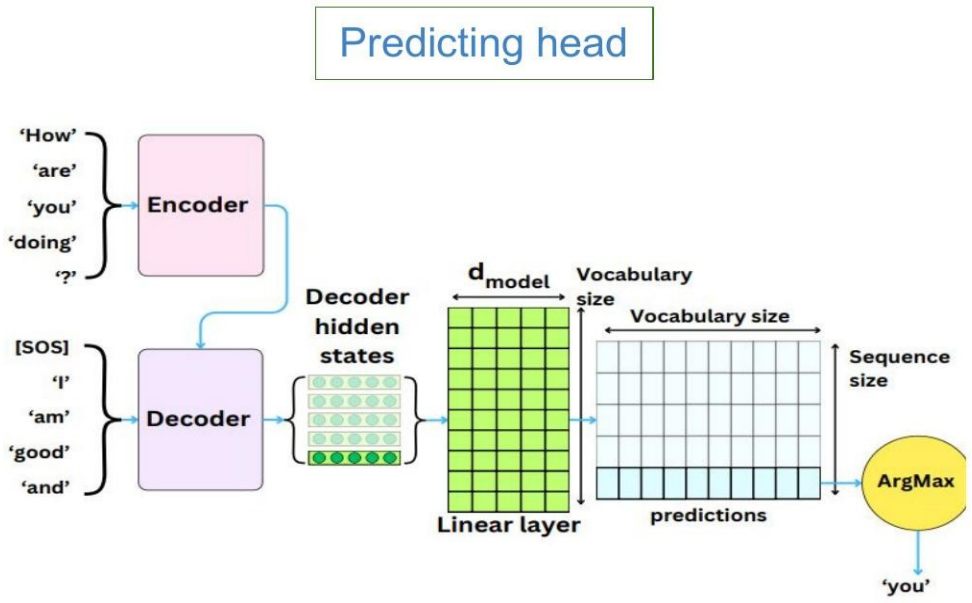

Figure 4.2 — The prediction head. Decoder hidden states are projected to vocabulary-size logits by a linear layer; ArgMax (or sampling) yields the predicted token. Here the model translates "How are you doing ?" into "I am good and ...".

4.2 Autoregressive Decoding — Step by Step

At inference time the decoder generates one token at a time. At step t, the input to the decoder is the [SOS] token followed by all previously generated tokens. The output of the decoder at position t is fed to the prediction head, the next token is chosen, appended to the decoder input, and the process repeats. Generation halts when the model emits the [EOS] (end-of-sequence) token.

t = 1 *First step*	Decoder input: <SOS> **Decoder output: Je**
t = 2 *Second step*	Decoder input: <SOS> Je **Decoder output: Je vais**
t = 3 *Third step*	Decoder input: <SOS> Je vais **Decoder output: Je vais bien**
t = 4 *Fourth step (halt)*	Decoder input: <SOS> Je vais bien **Decoder output: Je vais bien <EOS>** *<EOS> emitted → generation stops.*

Figure 4.3 — Autoregressive decoding for the translation "I am good" → "Je vais bien". The encoder representation (computed once) is reused at every step; the decoder generates one token at a time until <EOS>.

5. BERT — Bidirectional Encoder Representations from Transformers

5.1 What is BERT?

BERT (Devlin et al., 2018) takes the encoder half of the Transformer and trains it on a massive text corpus to produce deep bidirectional representations of language. The acronym stands for Bidirectional Encoder Representations from Transformers. "Bidirectional" means that when computing the representation of any token, BERT can attend to context on both the left and the right — this is fundamentally different from left-to-right language models like GPT.

BERT is a pre-trained Transformer encoder stack. After pre-training on raw text (no human labels), it can be fine-tuned on a small labeled dataset to solve almost any NLP task: sentiment analysis, named entity recognition, question answering, semantic textual similarity, and many more.

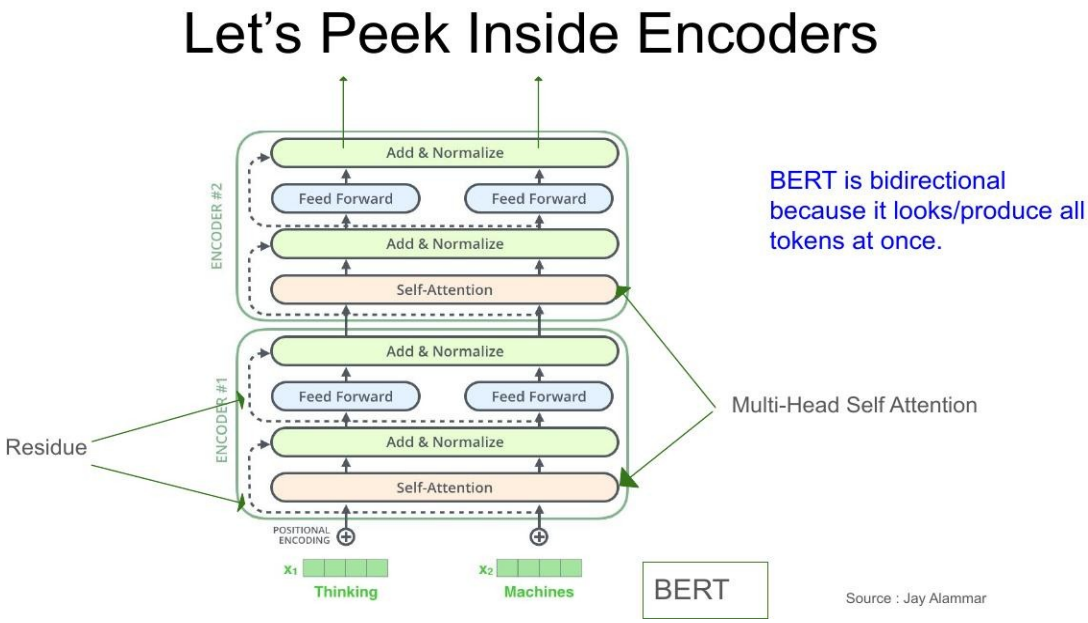

Figure 5.1 — BERT's bidirectionality. When computing the representation of "Thinking", the self-attention layer can attend to all other tokens (including "Machines" to its right) simultaneously.

5.2 BERT Configurations

The original BERT paper released two configurations:

Configuration	Architecture	Parameter count
BERT-base	12 layers, 768 hidden, 12 heads	*~110M parameters*
BERT-large	24 layers, 1024 hidden, 16 heads	*~340M parameters*

Subsequent work introduced smaller distilled variants — BERT-tiny, BERT-mini, BERT-small, BERT-medium — for resource-constrained settings.

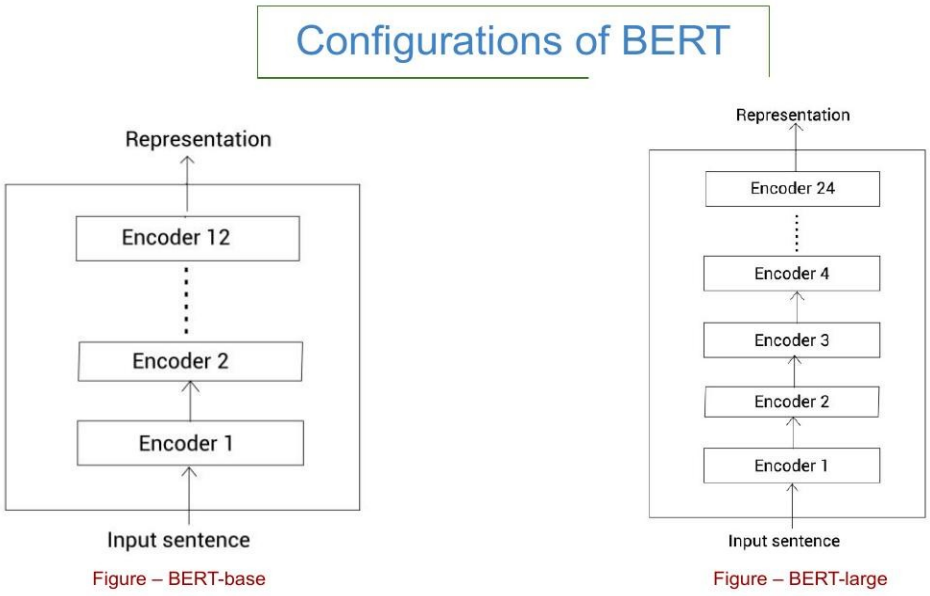

Figure 5.2 — BERT-base (12 encoders, left) vs. BERT-large (24 encoders, right). Both produce a contextual representation for every input token.

5.3 Pre-training Strategy

BERT is pre-trained on two unsupervised objectives simultaneously.

5.3.1 Masked Language Modeling (MLM)

BERT belongs to the family of auto-encoding language models — it is trained to reconstruct corrupted input rather than to predict the next token. Specifically, 15% of input tokens are randomly selected and "masked". BERT must predict the original identity of these masked tokens using bidirectional context.

Of the selected 15%:

- 80% are replaced by a special [MASK] token.
- 10% are replaced by a random token.
- 10% are left unchanged.

(This 80-10-10 rule prevents the model from over-relying on the [MASK] token, which never appears at fine-tuning or inference time.)

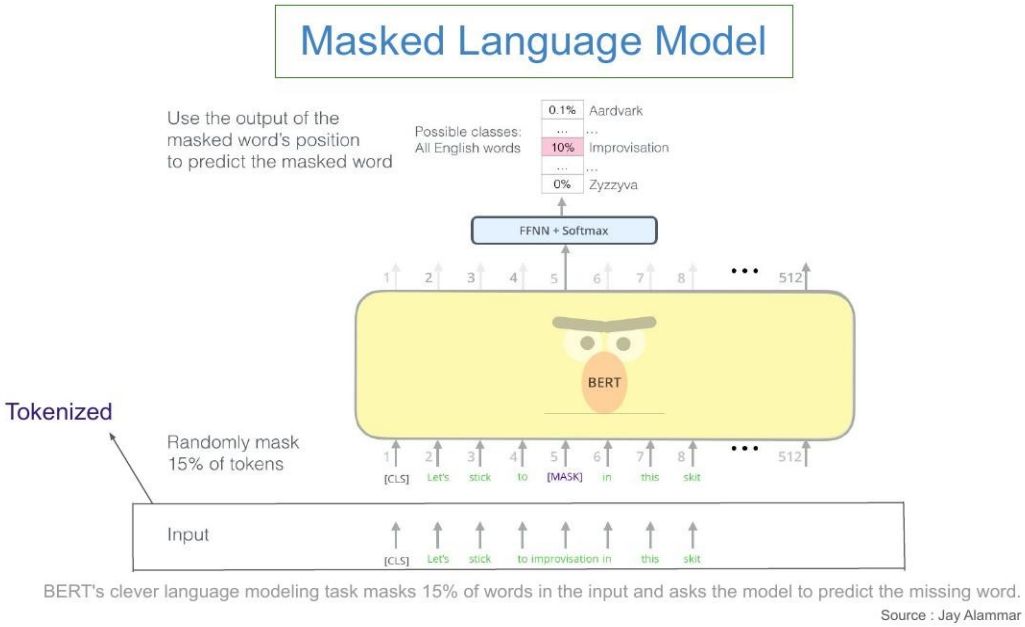

Figure 5.3 — Masked Language Modeling. The input "Let's stick to improvisation in this skit" has its 5th token replaced by [MASK]. The output at position 5 is fed through a feed-forward + softmax over the vocabulary; the loss is the cross-entropy between the predicted distribution and the true word "improvisation".

5.3.2 Next Sentence Prediction (NSP)

To capture sentence-level relationships (useful for QA and natural language inference), BERT is also trained on a binary classification task. Given two sentences A and B, predict whether B is the actual sentence that followed A in the original corpus, or a random unrelated sentence. The label uses the output corresponding to the special [CLS] token at the start of the input.

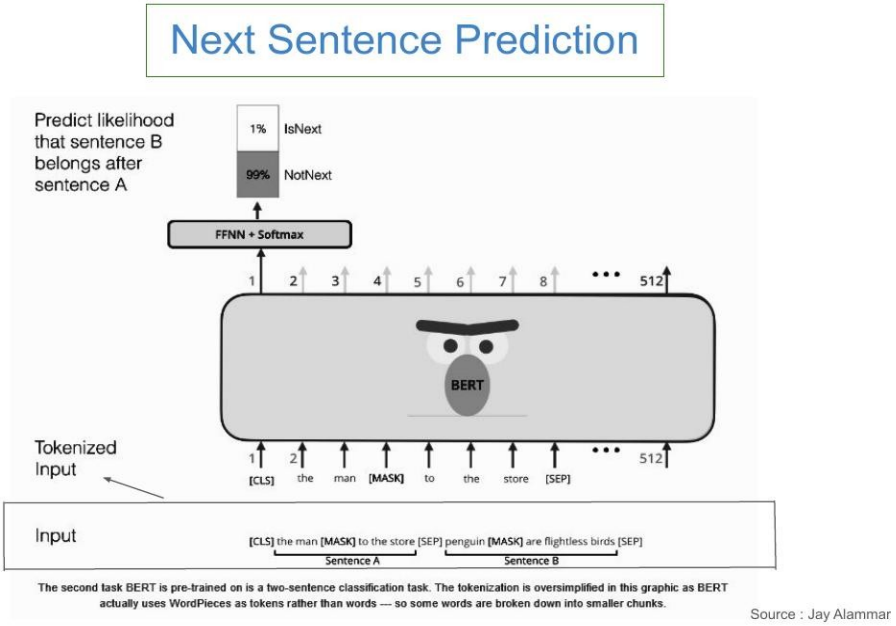

Figure 5.4 — Next Sentence Prediction. Two sentences are concatenated with [CLS] at the start and [SEP] tokens between/after. The [CLS] output is fed to a classifier predicting IsNext / NotNext.

5.4 Input Representation

Every input token embedding to BERT is a sum of three pieces:

$$\text{Input embedding} = \text{Token embedding} + \text{Segment embedding} + \text{Position embedding}$$

The token embedding is the WordPiece embedding. The segment embedding is one of two learned vectors (E_A, E_B) indicating which sentence the token belongs to (used in NSP). The position embedding is a learned vector for each absolute position. All three have the same dimension d_model and are added element-wise.

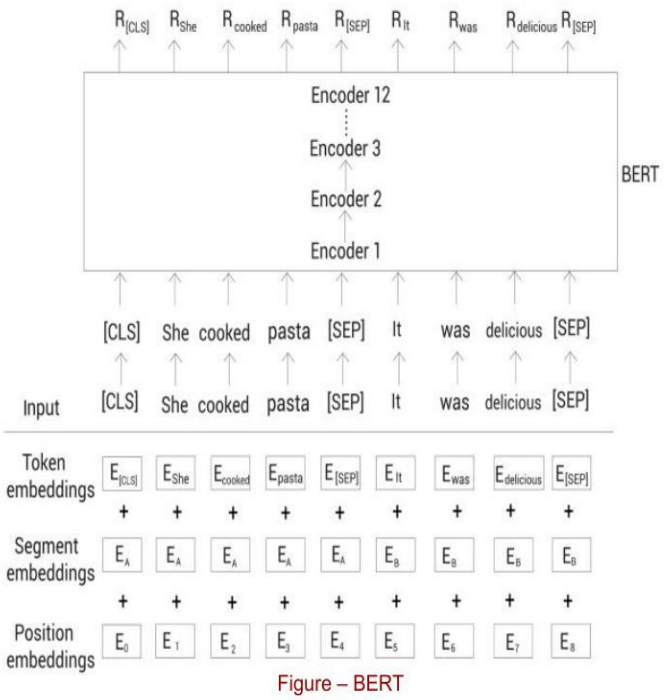

Figure 5.5 — BERT input representation for the pair "She cooked pasta" / "It was delicious". Special tokens [CLS] and [SEP] frame the input; the three embeddings (token, segment, position) are summed before entering encoder 1.

Why these two pre-training tasks together?

MLM forces BERT to learn token-level bidirectional context. NSP forces it to learn sentence-level relationships. Together they produce representations that are useful for both token-level tasks (NER, POS tagging) and sentence-level tasks (sentiment, NLI, QA). However, later work (RoBERTa, 2019) showed that NSP is actually unnecessary if MLM is trained on enough data with longer sequences — modern BERT variants often drop NSP entirely.

5.5 Fine-Tuning BERT

After pre-training, BERT can be fine-tuned for a downstream task by appending a small task-specific head (typically a single linear classifier) on top of the [CLS] output (for sentence classification) or on top of every token's output (for token-level tasks like NER). All BERT parameters are then trained jointly with the task head on the task's labeled data — usually for just a few epochs. This is the canonical "pretrain-then-finetune" recipe that has dominated NLP since 2018.

Note: at fine-tuning time we do NOT mask any tokens. Masking is only used during pre-training.

5.6 DistilBERT — Smaller, Faster, Almost as Good

DistilBERT (Sanh et al., 2019) is a lightweight version of BERT produced by HuggingFace using the technique of knowledge distillation. The original BERT-base ("teacher") is used to train a smaller student model ("DistilBERT") with half the layers (6 instead of 12). The student is trained to mimic the teacher's output distribution rather than just the hard labels.

DistilBERT retains roughly 97% of BERT's language understanding performance while being 40% smaller and 60% faster at inference. It is trained only on the MLM objective (no NSP).

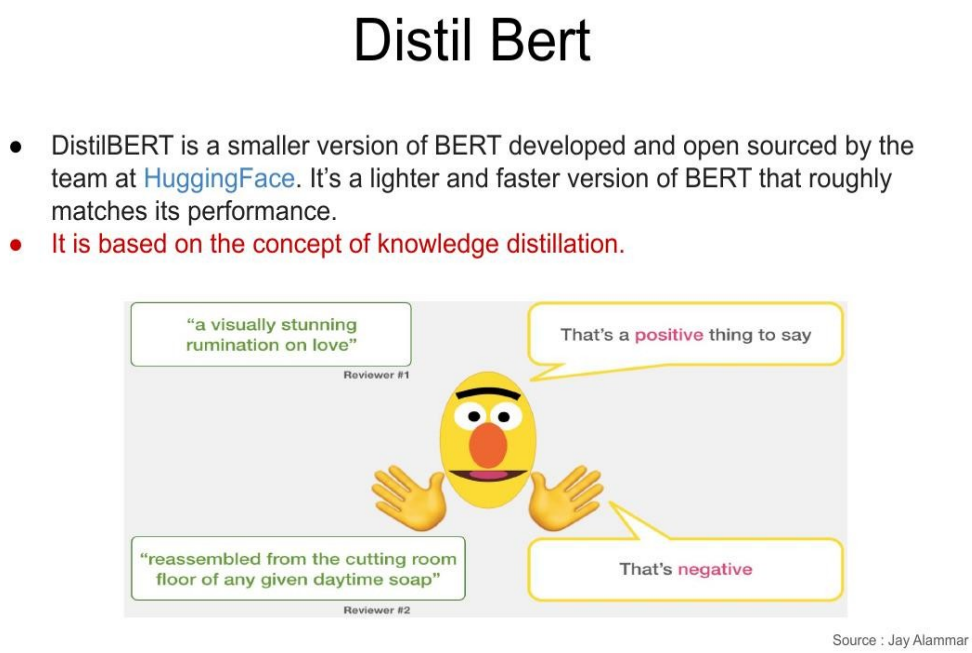

Figure 5.6 — DistilBERT applied to sentiment classification. The pre-trained DistilBERT processes the review text; only a small classifier on top of its [CLS] output is trained on the task's labeled data.

6. GPT — Generative Pre-trained Transformer

6.1 Decoder-Only Architecture

Where BERT uses the encoder half of the Transformer, GPT (Radford et al., 2018, 2019, 2020) uses the decoder half — but with one key simplification. Since GPT does not consume a separate source sentence, the cross-attention sub-layer is dropped entirely. Each GPT block contains only:

- Masked multi-head self-attention (causal: each token attends only to itself and earlier tokens)
- Position-wise feed-forward network
- Residual + Layer Norm wrappers

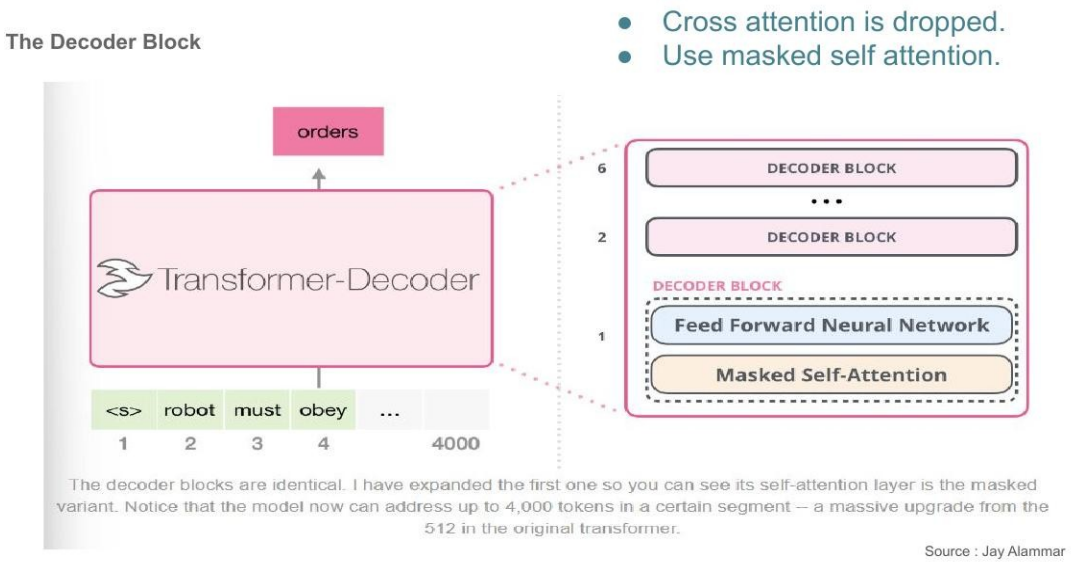

Figure 6.1 — GPT's decoder block. Cross-attention is dropped; the attention is causal (masked). GPT-2 uses 6 (small) to 48 (extra-large) such blocks, with context length 1024 originally and growing in subsequent versions.

6.2 Autoregressive Language Modeling

GPT is trained by maximum likelihood on raw text. Given a sequence of tokens ($x_1, x_2, ..., x_T$), the model maximises:

$$L = \Sigma_t \log p_\theta(x_t \mid x_1, x_2, ..., x_{t-1})$$

In words: predict each token given all preceding tokens. Because attention is causally masked, position t can only see positions 1 through t−1, exactly matching this objective. This is auto-regressive language modeling — the foundation of every generative LLM.

At inference, generation is performed token by token: feed the model a prompt, sample the next token from the output distribution, append it to the input, and repeat.

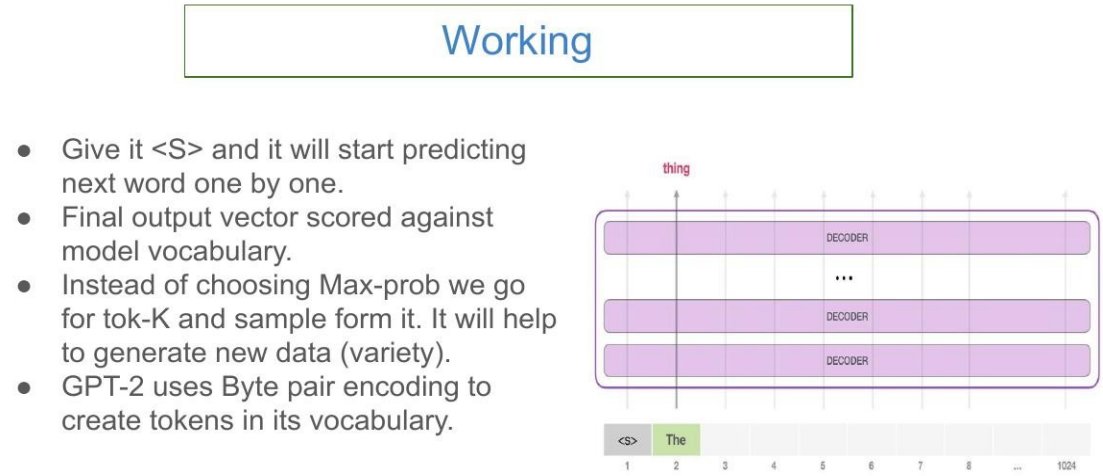

Figure 6.2 — GPT-2 generating text. Given the input <S> The, the model emits one token at a time ("thing" in this example) by feeding its previous output back as input.

6.3 Tokenization with Byte-Pair Encoding (BPE)

GPT-2 and GPT-3 use Byte-Pair Encoding (BPE), a sub-word tokenization scheme that strikes a balance between character-level and word-level tokenization. BPE starts with characters as tokens and iteratively merges the most frequent adjacent pair into a single token, building a vocabulary of common sub-word units (e.g., "ing", "tion", "un").

Advantages: (i) the vocabulary is finite and modest in size (~50,000 tokens for GPT-2); (ii) any string — including unseen words and emojis — can be encoded by falling back to characters; (iii) common words become single tokens while rare words decompose into meaningful sub-parts.

6.4 Sampling Strategies

Once the model produces a probability distribution over the next token, we must choose what to output. The simplest strategy is greedy decoding (always pick the argmax), but this produces repetitive, dull text. Better strategies include:

- Top-K sampling: keep the top-K most likely tokens, renormalise their probabilities, and sample. K = 40 is a common choice.
- Nucleus (Top-p) sampling: keep the smallest set of tokens whose cumulative probability exceeds p (e.g. p = 0.9), then sample. Adapts dynamically — at confident steps the set is small; at uncertain steps it grows.
- Temperature: divide logits by a temperature τ before softmax. $\tau < 1$ sharpens the distribution (more conservative); $\tau > 1$ flattens it (more diverse).

6.5 GPT-2 Sizes

The GPT-2 family demonstrated that simply scaling up an autoregressive Transformer produces dramatic capability gains. Later GPT-3 (175B parameters) and GPT-4 pushed this much further.

Model	Architecture	Parameters
GPT-2 Small	12 blocks, 768 hidden	*117M*
GPT-2 Medium	24 blocks, 1024 hidden	*345M*
GPT-2 Large	36 blocks, 1280 hidden	*762M*
GPT-2 Extra-Large	48 blocks, 1600 hidden	*1.5B*

6.6 The Self-Attention Cabinet Analogy

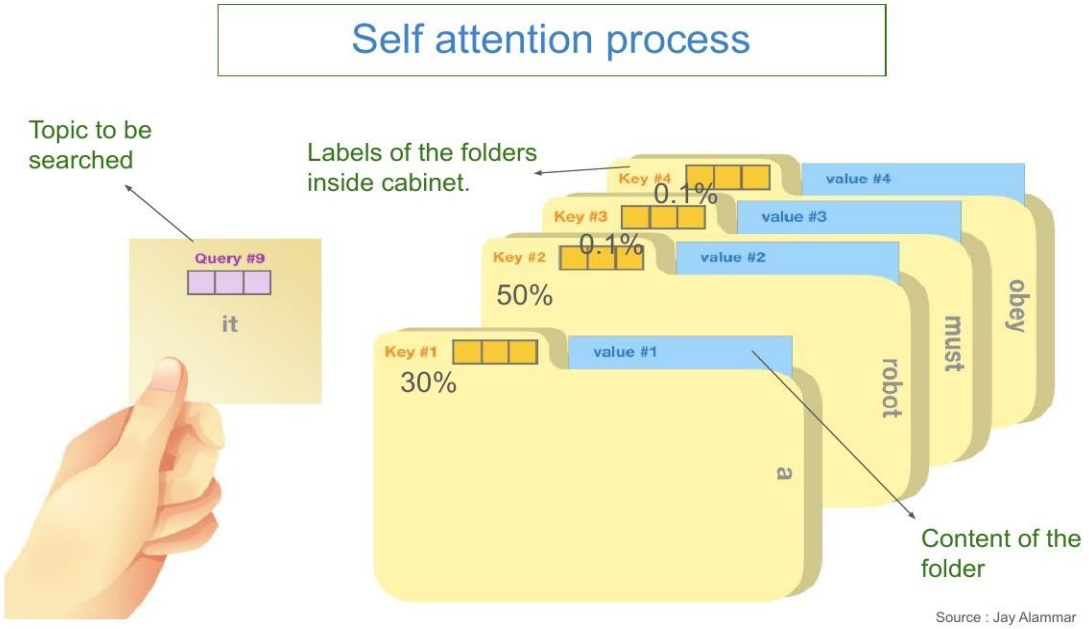

Figure 6.3 — Self-attention as a content-based filing cabinet. The query (the topic to be searched, here the word "it") is compared against every folder label (key); the resulting attention weights (50%, 30%, 0.1%, …) determine how much of each folder's content (value) is mixed into the output.

7. Large Language Models — Training Pipeline & RLHF

7.1 What Makes a Language Model "Large"?

A Large Language Model (LLM) is a Transformer-based language model with billions of parameters, trained on trillions of tokens of text using self-supervised learning. Examples include GPT-4, Claude, LLaMA, PaLM, Mistral, and Gemini. The key components that influence LLM architecture and capability are:

- Model size and parameter count (typically 7B → 70B → 175B → 1T+)
- Input representations (tokenizer choice, vocabulary size, context length)
- Self-attention mechanism (variants like Multi-Query Attention, Grouped-Query Attention, FlashAttention)
- Training objective (next-token prediction, possibly with auxiliary objectives)
- Computational efficiency (activation checkpointing, mixed-precision, model parallelism)
- Decoding strategy (greedy, top-K, top-p, beam search, speculative decoding)

7.2 The Four-Stage Training Pipeline

Modern conversational LLMs like ChatGPT and Claude are produced by a four-stage pipeline. Each stage takes the model produced by the previous stage and refines it using a different dataset and objective.

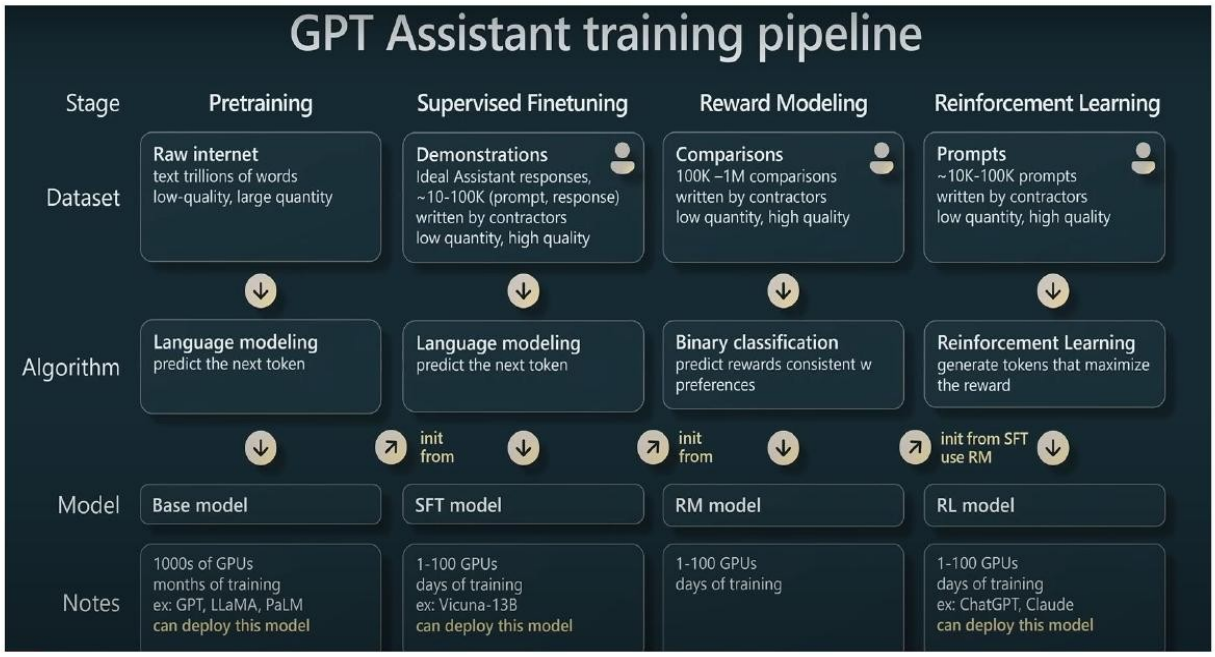

Figure 7.1 — The GPT-Assistant training pipeline (after Karpathy, 2023). Pretraining → Supervised Fine-Tuning → Reward Modeling → Reinforcement Learning. Each stage uses progressively smaller, higher-quality datasets and produces a more capable model.

Stage 1 — Pretraining

Train a Transformer LM from scratch on trillions of tokens of raw internet text using next-token prediction. This stage requires thousands of GPUs and months of training. The output is the base model — it knows a vast amount of world knowledge but is not yet helpful or aligned. Examples: GPT-3 base, LLaMA, PaLM.

Stage 2 — Supervised Fine-Tuning (SFT)

Take the base model and fine-tune it on a small, high-quality dataset of (prompt, ideal response) pairs written by human contractors (10K–100K examples). The objective is the same — next-token prediction — but the data shifts the model from "text completion" to "helpful assistant." Output: SFT model. Examples: Vicuna, instruction-tuned LLaMA. Requires only 1–100 GPUs, days of training.

Stage 3 — Reward Modeling (RM)

Generate multiple responses from the SFT model for many prompts. Have human contractors rank pairs of responses by quality ("answer A is better than answer B"). Train a separate reward model $R_\varphi(\text{prompt}, \text{response}) \to \mathbb{R}$ to predict these human preference scores. The reward model learns to score responses the way humans do — capturing notions of helpfulness, accuracy, and safety that are hard to specify by hand.

Stage 4 — Reinforcement Learning (RL)

Initialise from the SFT model. For each prompt, sample a response from the policy, score it using the reward model, and update the policy to maximise expected reward — typically using PPO (Proximal Policy Optimization). A KL-divergence penalty keeps the RL policy close to the SFT model to prevent it from drifting into reward-model exploitation. The output is the final RL model — what gets deployed as ChatGPT, Claude, etc.

7.3 Reinforcement Learning from Human Feedback (RLHF)

Stages 3 and 4 together are called RLHF. Without RLHF, LLMs produce technically-fluent but often unhelpful, untruthful, or unsafe text. RLHF is the technique that takes a raw language model and turns it into a useful assistant aligned with human preferences.

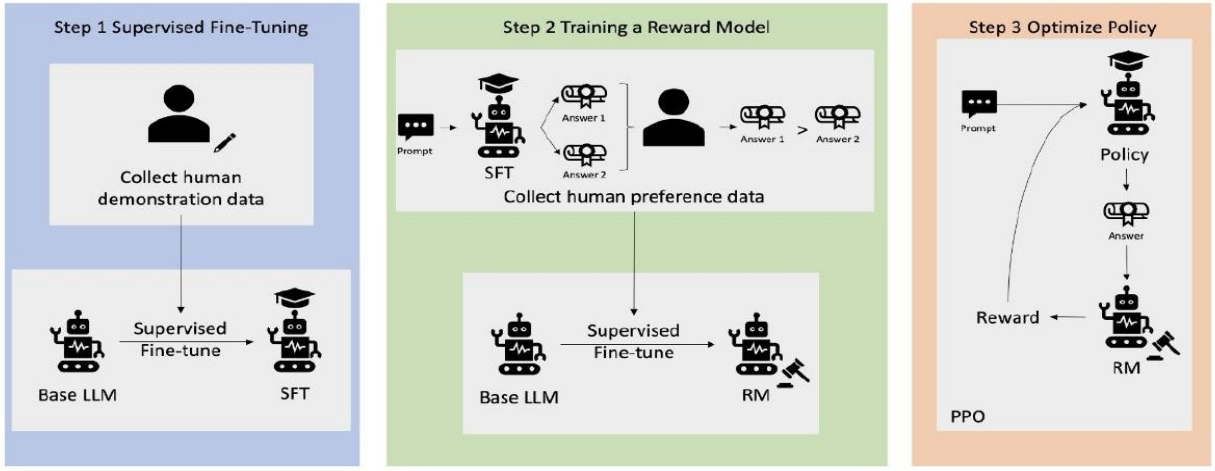

Figure : overview of the RLHF learning process

Figure 7.2 — Overview of the RLHF learning process. Step 1: collect human demonstrations and supervised-finetune the base LLM. Step 2: collect human pairwise preferences and train a reward model. Step 3: use PPO to optimise the SFT policy against the reward model.

Why a learned reward model rather than direct human feedback?

In principle one could ask humans to score every model output during training. In practice this is impossibly slow — RLHF needs millions of reward evaluations during PPO. The reward model is a learned proxy: trained once from a fixed dataset of human preferences, it can then provide unlimited fast feedback during the RL stage. The trade-off is reward hacking — the policy may learn to exploit quirks of the reward model that don't reflect true human preferences. This is mitigated by the KL penalty against the SFT policy and by ongoing dataset curation.

7.4 The PPO Objective

The reinforcement learning step uses Proximal Policy Optimization (PPO). The objective for a prompt-response pair (x, y) is:

$$J(\pi_\theta) = \mathbb{E}_{(x,y)\sim\pi_\theta}\left[R_\varphi(x,y) - \beta \cdot \mathrm{KL}\left(\pi_\theta(\cdot \mid x) \,\|\, \pi_{\mathrm{SFT}}(\cdot \mid x)\right)\right]$$

The first term rewards responses the reward model rates highly. The second term — the KL divergence between the current policy π_θ and the frozen SFT policy π_{SFT} — penalises the policy for drifting too far from sensible language. The coefficient β controls this trade-off; typical values are $\beta \approx 0.01$ – 0.1.

8. Large Vision Models (LVMs)

8.1 What Are LVMs?

Large Vision Models (LVMs) are advanced models that tackle a wide range of visual tasks — image classification, object detection, segmentation, image captioning, visual question answering, and more. Like LLMs, they typically involve hundreds of millions to billions of parameters and are pre-trained on large general-purpose image datasets (e.g. ImageNet, LAION-5B, WebImageText) before being fine-tuned for specialised downstream tasks.

Prominent examples include Google's Vision Transformer (ViT), OpenAI's CLIP (Contrastive Language–Image Pretraining), OpenAI's DALL·E and GPT-4o, and DINO (self-distillation with no labels).

8.2 Evolution of LVMs

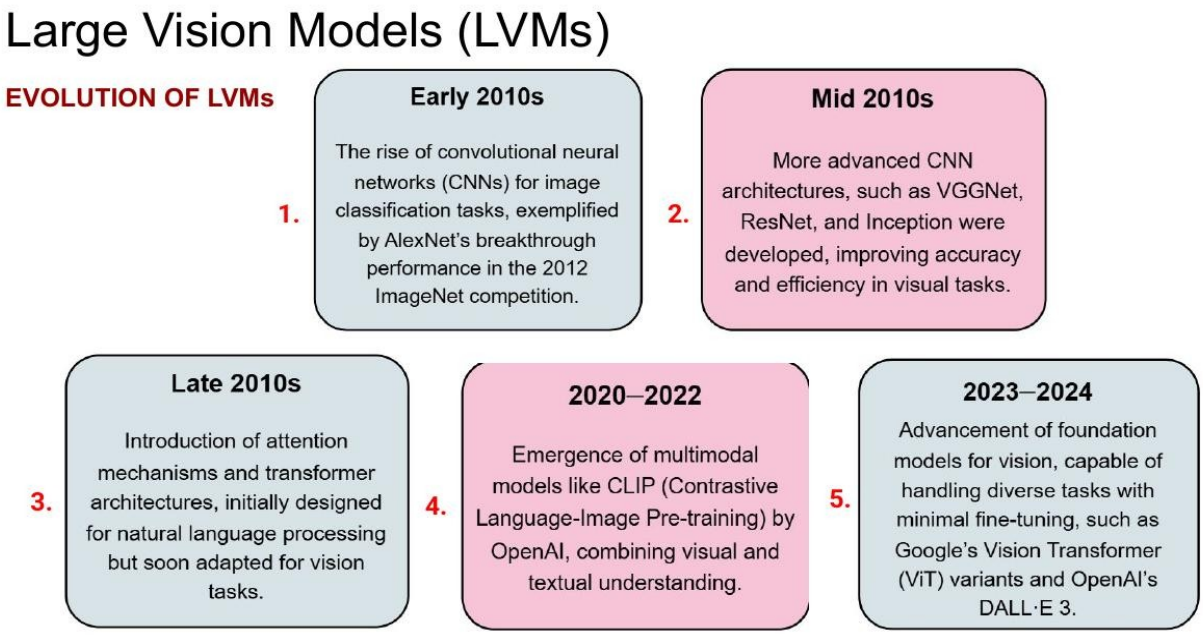

Figure 8.1 — Evolution of large vision models. (1) Early 2010s: AlexNet (2012) sparks the CNN era. (2) Mid-2010s: VGGNet, ResNet, Inception refine CNN architectures. (3) Late 2010s: attention mechanisms originally from NLP migrate to vision. (4) 2020–22: multimodal models like CLIP unify vision and language. (5) 2023–24: foundation vision models (ViT variants, DALLE 3) handle diverse tasks with minimal fine-tuning.

8.3 LLMs vs LVMs

Aspect	LLM	LVM
Input modality	Sequences of text tokens	*Pixels / patches of images*
Backbone	Transformer decoder (GPT)	*Transformer encoder (ViT) or CNN*
Pretraining	Next-token prediction on text	*Image classification, contrastive, or self-supervised on image (or*

		image+text) corpora
Examples	GPT-4, Claude, LLaMA, PaLM 2	*ViT, CLIP, DINO, GPT-4o (vision), DALLE*

8.4 Convergence of Language and Vision

The boundary between LLMs and LVMs is dissolving. Several key developments mark the convergence:

- Multimodal models like CLIP align text and image representations in a shared embedding space, enabling zero-shot image classification from text prompts. DALL·E and Stable Diffusion go the opposite direction — text-to-image generation.
- Visual question answering models combine image encoders with language decoders to answer natural-language questions about images.
- Video understanding extends these architectures with temporal modelling for clip captioning and action recognition.
- Cross-modal transfer learning leverages knowledge learned in one modality to improve the other — e.g., language supervision improving image classification.

Today's frontier multimodal models (GPT-4o, Gemini, Claude with vision) accept text, images, audio, and video in a single unified token stream — a true generalisation of the Transformer to all modalities.

9. Vision Transformer (ViT)

9.1 Motivation

From AlexNet (2012) onwards, computer vision was dominated by convolutional neural networks — VGG, ResNet, Inception, EfficientNet. The Vision Transformer (Dosovitskiy et al., 2020), introduced in the paper "An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale," was the first work to successfully train a pure Transformer on ImageNet at competitive accuracy.

Key empirical finding: ViT beats CNNs only when pretraining data is sufficiently large. With ImageNet alone, ResNets win. Pre-train on ~100M images (e.g. JFT-300M) and ViT begins to win. Pre-train on 300M+ and ViT clearly wins. CNNs have strong inductive biases (locality, translation equivariance) that help with small data; Transformers have weaker biases and need more data to overcome them — but once they have it, they scale further.

9.2 ViT Architecture

The recipe for ViT is conceptually simple. Treat an image as a sequence of patches and feed it to a standard Transformer encoder. Concretely:

Step 1 *Split into patches*	Split the H×W×C input image into a grid of fixed-size patches (typically 16×16 pixels each, with stride 16 — non-overlapping). For a 224×224 image, this yields N = 14·14 = 196 patches.
Step 2 *Flatten each patch*	Flatten each patch from shape (h, w, c) to a vector of length h·w·c. For a 16×16×3 RGB patch this is a 768-dimensional vector.
Step 3 *Linear projection (embedding)*	Project each flattened patch through a learned linear layer ("patch embedding") of shape (h·w·c) × D, where D is the model dimension. Result: a sequence of N embedded patches z_1, z_2, …, z_N ∈ ℝ^D.
Step 4 *Prepend [CLS] token*	Prepend a randomly initialised, learnable [CLS] token vector z_0 to the sequence. Its job is to aggregate information from all patches into a single classification representation. Final sequence length: N + 1.
Step 5 *Add positional encoding*	Add learned 1-D positional embeddings (one per position 0, 1, …, N) to the patch embeddings. This injects spatial location information.
Step 6 *Pass through Transformer encoder*	The N+1 embedded tokens are passed through L identical Transformer encoder blocks (multi-head self-attention + MLP + Add&Norm — exactly as in the original Transformer). Output: N+1 contextual vectors c_0, c_1, …, c_N.
Step 7 *Classification head on c_0*	The output corresponding to the [CLS] token (c_0) is fed through a classification head (an MLP at pre-training time, a single linear layer at fine-tuning time) followed by softmax to produce class probabilities.

Table 9.1 — The seven-step ViT pipeline for image classification.

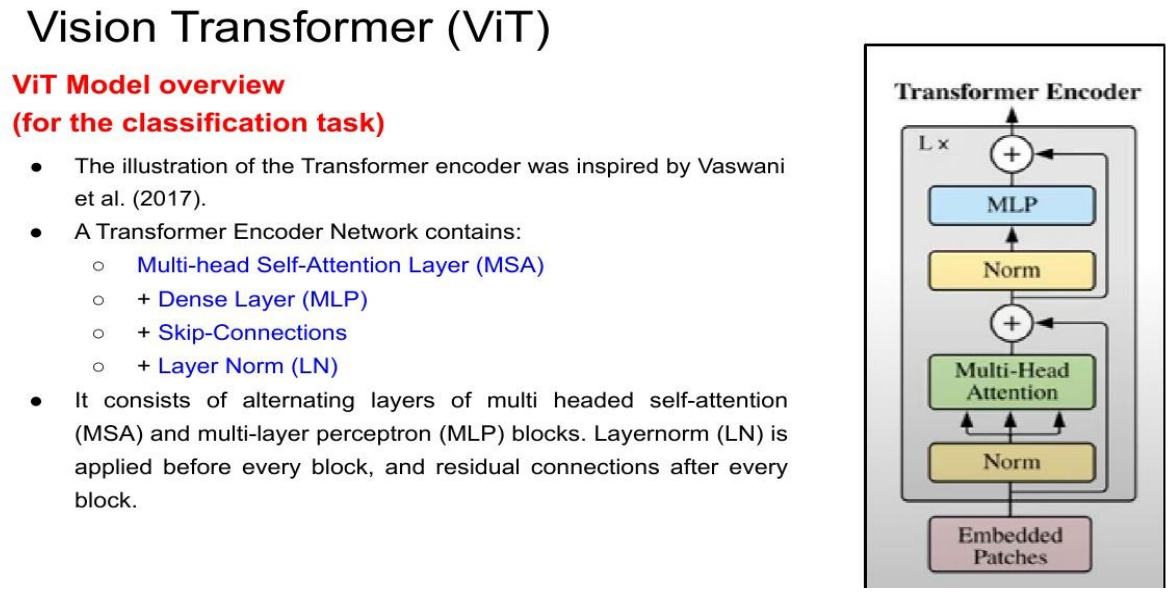

Figure 9.1 — A single ViT Transformer encoder block. Architecturally identical to the original NLP encoder: Norm → Multi-Head Attention → Add → Norm → MLP → Add. Pre-norm (LayerNorm before each sub-layer) is used in ViT, slightly different from the post-norm of the original Transformer.

9.3 The Patch Embedding Step

The crucial operation that adapts a Transformer to images is the patch embedding. The image is partitioned into a grid of non-overlapping patches; each patch is flattened to a vector and linearly projected to the model dimension. This sequence of patch embeddings is exactly analogous to the sequence of word embeddings in NLP — and the same Transformer encoder consumes them.

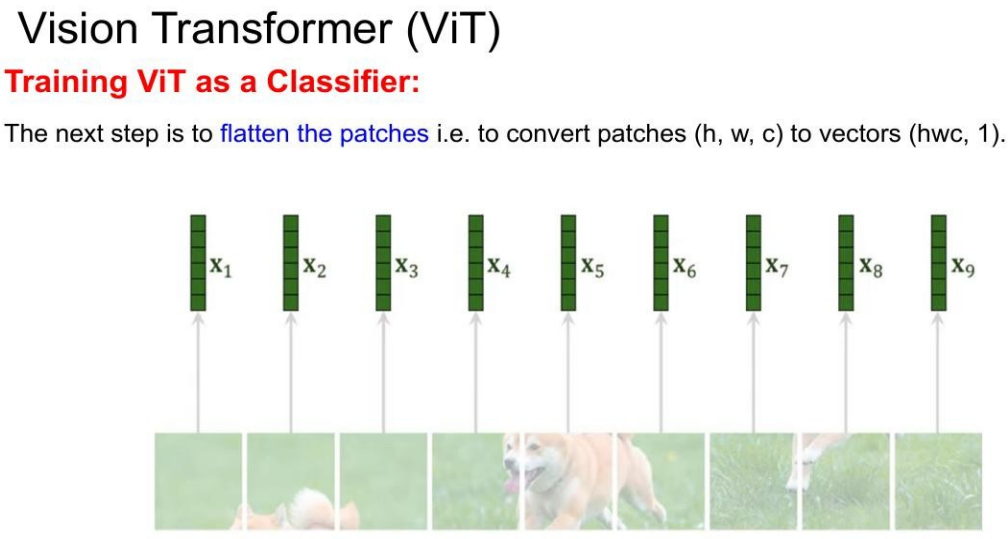

Figure 9.2 — Patch flattening. The dog image is split into 9 patches; each patch is flattened to a vector x_i. These vectors are then linearly projected to D-dimensional patch embeddings.

9.4 The [CLS] Token

Borrowed directly from BERT, the [CLS] token is a learnable vector prepended to the sequence of patch embeddings. Through self-attention layers, the [CLS] token "queries" every patch and aggregates a global summary of the image. After L Transformer layers, only the output corresponding to [CLS] is used for classification — $c_0 \in \mathbb{R}^D$, fed through a softmax head over class probabilities.

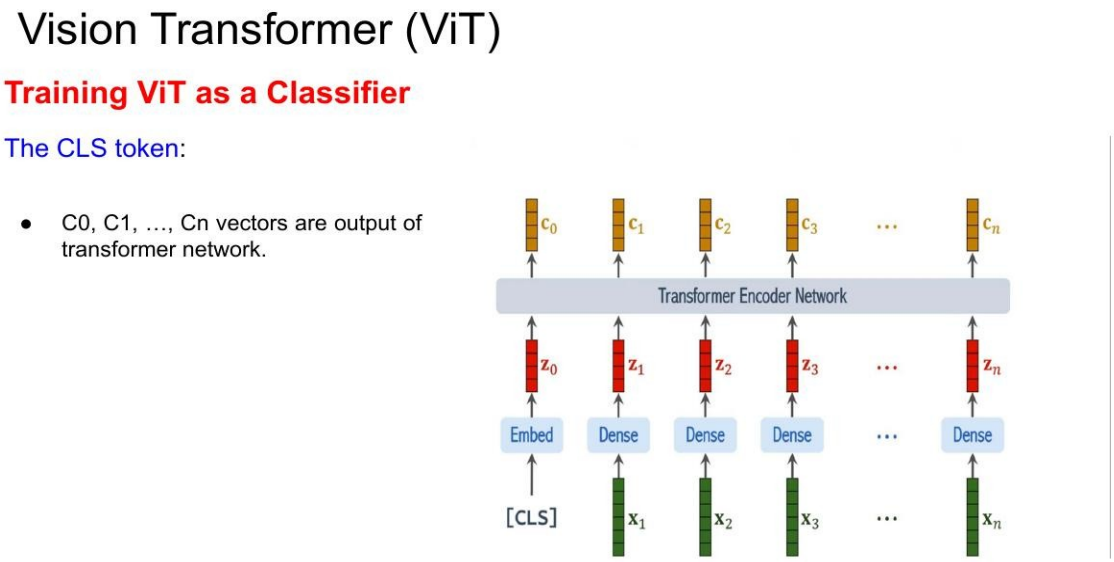

Figure 9.3 — The [CLS] token pipeline. Patch embeddings z_1, …, z_n + the [CLS] embedding z_0 are passed through the Transformer encoder. The output c_0 is the image representation used for classification; c_1, …, c_n are not used at the classification head.

9.5 Pretraining and Fine-tuning Recipe

Vision Transformer (ViT)

Training ViT as a Classifier (Complete Training)

- Randomly initialize the model.
- Train the model on some large image dataset A. This will give a pretrained model.
- Fine-tune the model on the training dataset.
- Evaluate the dataset on the test-set of dataset B.

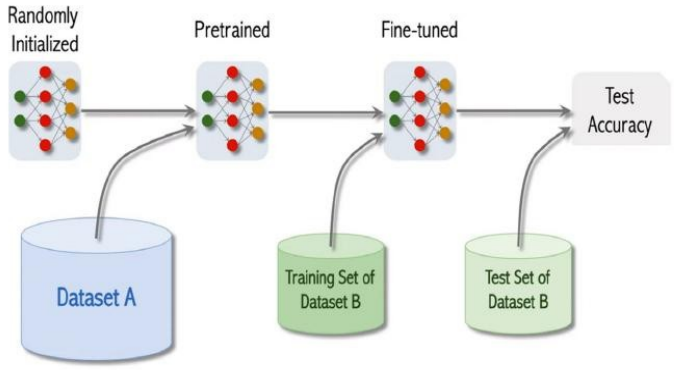

Figure 9.4 — ViT training pipeline. Randomly initialise → pretrain on large dataset A (e.g. ImageNet-21k or JFT-300M) → fine-tune on the target downstream training set (dataset B) → evaluate on dataset B's test set. Fine-tuning typically replaces the multi-layer MLP classification head with a single linear layer.

10. DINO — Self-Distillation with No Labels

10.1 Why Self-Supervised Pretraining for ViTs?

The ViT recipe of supervised pretraining on ImageNet-21k or JFT-300M requires labels for hundreds of millions of images — an enormous human-labelling effort. DINO (Caron et al., ICCV 2021) explores whether ViTs can be pretrained without any labels, using self-supervision.

In supervised learning we have data pairs (x_i, y_i) where y_i is annotated by humans. In self-supervised learning we still have x_i, but y_i is replaced by a pseudo-label p_i generated automatically from x_i itself by a pretext task. The pretext task is designed so that solving it forces the model to learn rich intermediate representations.

10.2 The DINO Idea — Self-Distillation

DINO uses two networks of identical architecture: a student network g_θ_s and a teacher network g_θ_t. Both take views of the same image and produce probability distributions over a fixed set of features ("prototypes").

The training procedure for one image:

- Generate two augmented views (random crops, color jitter, etc.) of the same image.
- Pass both views through both networks. The teacher produces target distributions; the student produces predicted distributions.
- Minimise the cross-entropy from teacher distribution to student distribution.
- Update only the student parameters by gradient descent. The teacher parameters are an exponential moving average (EMA) of the student parameters — no gradients flow through the teacher.

This is called self-distillation because the model effectively distils knowledge from a slowly-evolving copy of itself. There are no human labels anywhere — the teacher provides pseudo-labels that the student learns to match.

10.3 Lineage — BYOL and PBL

DINO is inspired by BYOL (Bootstrap Your Own Latent, Grill et al., 2020), which in turn was inspired by PBL (Predictions of Bootstrapped Latents, Guo et al., 2020) from deep reinforcement learning. The common idea — two networks learning from each other through bootstrapped predictions, no negative samples needed — has its roots in deep RL representation learning.

Method	Idea	Reference
DINO (2021)	Self-distillation with no labels — applies BYOL-style learning to ViT	*Caron et al., ICCV 2021*
BYOL (2020)	Two networks (online & target) learn image representations without negatives	*Grill et al., NeurIPS 2020*
PBL (2020)	Predictions of bootstrapped latents — multitask RL representation learning	*Guo et al., ICML 2020*

Deep RL	Original inspiration — target networks in DQN, slowly-updated value networks	*Mnih et al., 2015*

10.4 Why DINO Matters

Two findings from DINO have shaped self-supervised learning:

- DINO-pretrained ViTs implicitly learn semantic segmentation. Visualising the [CLS] token's attention map reveals that DINO-trained models attend to object boundaries even though they were never trained on segmentation labels — an emergent property of self-supervised ViTs.
- DINO features transfer well to downstream tasks (k-NN classification on ImageNet, image retrieval, copy detection) often matching or beating supervised pretraining — without ever using a human label.

This established self-supervised pretraining of ViTs as a viable, label-free alternative to supervised pretraining, and influenced subsequent foundation models (DINOv2, SAM, etc.).

11. Summary, Symbol Reference & Practical Notes

11.1 Summary at a Glance

The Transformer family in one screen

Transformer (2017): Encoder-Decoder. Self-attention replaces recurrence. Foundation for everything that followed.

BERT (2018): Encoder-only. Bidirectional. Pretrained with MLM + NSP. The dominant model for NLU until ~2022.

GPT (2018−): Decoder-only. Autoregressive. Pretrained with next-token prediction. Foundation of all generative LLMs.

RLHF: SFT → RM → PPO. Aligns base LLMs with human preferences. The technique behind ChatGPT, Claude, etc.

ViT (2020): Transformer encoder applied to image patches. Beats CNNs at scale. Foundation of modern vision models.

DINO (2021): Self-supervised pretraining for ViT via teacher-student distillation. No labels needed.

11.2 Symbol Reference Table

Symbol	Meaning	Type / Range
Q, K, V	Query, Key, Value matrices	*Each* $\in \mathbb{R}^{n \times d_k}$ *or* $\mathbb{R}^{n \times d_v}$
d_model	Model hidden dimension (e.g. 512, 768, 1024)	*Scalar*
d_k, d_v	Per-head key & value dimensions (e.g. 64)	*Scalar*
d_ff	FFN inner dimension (typically 4 × d_model)	*Scalar*
h	Number of attention heads (e.g. 8, 12, 16)	*Scalar*
N or L	Number of encoder/decoder blocks (e.g. 6, 12, 24)	*Scalar*
n	Sequence length / number of tokens or patches	*Scalar*
W^Q, W^K, W^V	Learned linear projections producing Q, K, V from X	*Matrices*
W^O	Output projection after multi-head concat	*Matrix*
PE(pos, i)	Sinusoidal positional encoding at position pos, dim i	*Scalar in [-1, 1]*
[CLS]	Special classification token prepended to input	*Learnable vector*
[SEP]	Sentence separator token (BERT)	*Learnable vector*
[MASK]	Masking token used during BERT MLM pretraining	*Learnable vector*
π_θ	Policy in RLHF (the language model itself)	*Distribution over tokens*
$R_\varphi(x, y)$	Reward model output for prompt x, response y	*Scalar reward*

β (KL coeff)	PPO regulariser strength against SFT policy	*Typically 0.01–0.1*
g_θ_s, g_θ_t	Student and teacher networks in DINO	*Identical architecture*

11.3 Practical Hyperparameter Defaults

Model	Configuration	Source
Original Transformer	d_model=512, h=8, N=6, d_ff=2048	*Vaswani et al. 2017*
BERT-base	d_model=768, h=12, L=12, d_ff=3072	*Devlin et al. 2018*
BERT-large	d_model=1024, h=16, L=24, d_ff=4096	*Devlin et al. 2018*
GPT-2 small	d_model=768, h=12, L=12, ctx=1024	*Radford et al. 2019*
GPT-2 XL	d_model=1600, h=25, L=48, ctx=1024	*Radford et al. 2019*
ViT-Base/16	d_model=768, h=12, L=12, patch=16×16	*Dosovitskiy et al. 2020*

11.4 Common Pitfalls

- Forgetting the $\sqrt{d_k}$ scaling: without it, softmax saturates and gradients vanish for large d_model.
- Forgetting positional encoding: without it, the Transformer is permutation-equivariant and cannot learn order-sensitive tasks.
- Using BatchNorm instead of LayerNorm: LayerNorm is essential because it works per-token and tolerates variable batch sizes / sequence lengths.
- Decoder unmasking bug: at training time, omitting the causal mask causes the model to cheat by looking at future tokens — the model trains fine but is useless at inference.
- Fine-tuning BERT with [MASK]: never mask tokens during fine-tuning; masking is only for pretraining.
- RLHF reward hacking: too small a KL coefficient β lets the policy drift into reward-model exploitation, producing fluent but nonsensical text. Monitor KL divergence during PPO training.
- ViT with too little data: ViT requires ~100M+ pretraining images to beat CNNs. On small datasets (CIFAR, small ImageNet subsets), use a pretrained ViT or stick with a ResNet.

11.5 References

Vaswani et al. (2017). Attention Is All You Need. NeurIPS.

Devlin et al. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL.

Radford et al. (2018). Improving Language Understanding by Generative Pre-Training. (GPT-1)

Radford et al. (2019). Language Models are Unsupervised Multitask Learners. (GPT-2)

Brown et al. (2020). Language Models are Few-Shot Learners. NeurIPS. (GPT-3)

Sanh et al. (2019). DistilBERT, a distilled version of BERT. NeurIPS Workshop.

Christiano et al. (2017). Deep Reinforcement Learning from Human Preferences. NeurIPS.

Ouyang et al. (2022). Training language models to follow instructions with human feedback. NeurIPS. (InstructGPT)

Dosovitskiy et al. (2020). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. ICLR.

Caron et al. (2021). Emerging Properties in Self-Supervised Vision Transformers. ICCV. (DINO)

Grill et al. (2020). Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning. NeurIPS. (BYOL)